



Universidad
Indoamérica

ユニット 3・オペレーティングシステムの基礎

ファイルとファイルシステム

Universidad Tecnológica Indoamérica 演習 TD2

ファイルにはどうアクセスする？2つの層

ディスク上（永続）

- ファイルシステムの構造：
スーパーブロック、inode 表、ブロック
- inode：各ファイルのメタデータ
- ディレクトリ：名前 → *inode* のリスト

カーネル内（開いている間）

- ディスクリプタ表（プロセスごとに1つ）
- オープンファイル表（システム全体で1つ）
- メモリ上の inode 表（とページキャッシュ）

ファイルシステムの構造

GPT

EFI パーティション

スーパーブロック

ビットマップ
(空き領域)

inode 表

データブロック (とディレクトリ)

NVMe ディスク →

ext4: ブロックグループがパーティション全体で繰り返される (スーパーブロックのコピー付き)

実際の出力

```
$ df -iT . | cut -c1-80; stat -f -c 'block size: %S bytes' .  
Filesystem      Type  Inodes   IUsed   IFree IUse% Mounted on  
/dev/nvme0n1p5 ext4 25600000 2372366 23227634   10% /  
block size: 4096 bytes
```

ファイル = 内容 + inode

- **inode** が持つもの：種類と権限、ハードリンク数、サイズ、デバイス、UID/GID、タイムスタンプ、ブロックの場所
- 名前は持っていない：名前はディレクトリにある
- **元資料の修正**： `ctime` は作成時刻ではなく、最後の *inode* の変更時刻です。作成時刻 (*Birth*) は ext4 にあり、 `statx()` で読みます。

実際の出力

```
$ stat notas.txt
  File: notas.txt
  size: 11          Blocks: 8          IO Block: 4096
regular file
Device: 259,5    Inode: 12190686    Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/student)   Gid: (
1000/student)
Access: 2026-09-30 13:12:49.872600645 -0500
Modify: 2026-09-30 13:12:49.872600645 -0500
Change: 2026-09-30 13:12:49.879601063 -0500
 Birth: 2026-09-30 13:12:49.872600645 -0500
```

inode からブロックへ：ポインタとエクステント

inode → extent tree (depth 1, 340 entries per block)

	論理	長さ	物理
#0	0	32,768	34,816
#1	32,768	32,768	67,681
#2	65,536	32,768	100,546
#3	98,304	32,768	133,411
#4	131,072	32,768	166,276
#5	163,840	9,111	199,141

物理ブロック =

$199,141 + (172,950 - 163,840)$

= 208,251

読んだ索引ブロック：1

同じファイルをポインタで (ext3) :

172,951 個のポインタが 170 個の索引ブロックに ・ この位置：索引の読み込み 2 回・ エクステントなら：6 × 12 バイト

676 MiB

論理ブロックで見たファイル (0 ... 172,951)。各帯が1つのエクステント

名前は inode を指す

実際の出力

```
$ ln notas.txt copia.txt; ln -s notas.txt atajo; ls -li notas.txt copia.txt atajo
12190687 lrwxrwxrwx 1 student student  9 Sep 30 13:12 atajo → notas.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 copia.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 notas.txt
```

ハードリンク (`link`)

同じ inode への別のエントリ：番号も同じで `nlink = 2`。ディレクトリには作れず、ファイルシステムもまたげない。

シンボリックリンク (`symlink`)

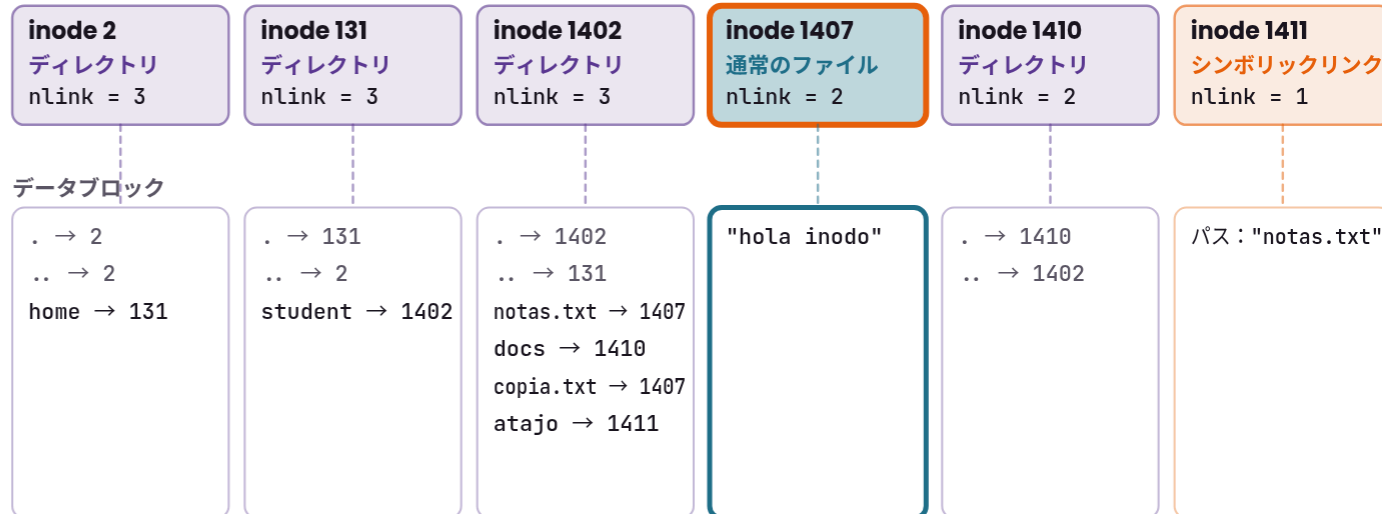
内容がパスである新しい inode (9 バイト：`notas.txt`)。宙に浮くことがある。

パスの解決、リンク、削除

\$ cat atajo

相対パス：カレントディレクトリ (inode 1402) から始まる
ディレクトリ 1402 を読む："atajo" → inode 1411
シンボリックリンク #1："notas.txt" で続ける
ディレクトリ 1402 を読む："notas.txt" → inode 1407
inode 1407 のデータブロックを読む："hola inodo"

inode 表



"hola inodo"

inode を読む： stat、fstat、lstat

C

```
int stat(const char *path, struct stat *st);
int fstat(int fd, struct stat *st);
int lstat(const char *path, struct stat *st); /* does not follow links */

struct stat {
    dev_t      st_dev;      ino_t    st_ino;
    mode_t      st_mode;    nlink_t st_nlink;
    uid_t      st_uid;      gid_t    st_gid;
    dev_t      st_rdev;     /* device, if special file */
    off_t      st_size;     blkcnt_t st_blocks;
    blksize_t st_blksize; /* preferred I/O block size */
    struct timespec st_atim, st_mtim, st_ctim;
};
```

- 成功なら 0、そうでなければ -1 と `errno` :
- `EACCES` パス上のディレクトリに検索権 (x) がない
- `ENOENT` 存在しない・`ENOTDIR` 途中の要素がディレクトリではない
- `ELoop` シンボリックリンクが 40 を超える
- 2026 年：ナノ秒単位の時刻 (`st_mtime = st_mtim.tv_sec`)。 `statx()` で作成時刻も得られる。

種類は `st_mode` に： `S_IS...` マクロ

<code>S_ISREG</code>	通常のファイル	-
<code>S_ISDIR</code>	ディレクトリ	d
<code>S_ISLNK</code>	シンボリックリンク	l
<code>S_ISCHR</code> ·	キャラクタ・ブロック	c ·
<code>S_ISBLK</code>	デバイス	b
<code>S_ISFIFO</code> ·	パイプ・ソケット	p ·
<code>S_ISSOCK</code>		s

実際の出力

```
$ ./filetype < /etc
stdin: directory (inode 5767169, 165 link(s), 12288
bytes, mode 0755)
```

実際の出力

```
$ echo hi | ./filetype
stdin: FIFO (pipe) (inode 859154, 1 link(s), 0 bytes,
mode 0600)
```

実際の出力

```
$ ./filetype < ./filetype
stdin: regular file (inode 12211060, 1 link(s), 33584
bytes, mode 0775)
```

権限： `access` ...とその落とし穴

C

```
int access(const char *path, int mode); /* R_OK W_OK X_OK F_OK */
```

- 存在しないか権限がなければ -1: `EROFS`
(読み取り専用)、`ETXTBSY` (実行中のファイル) ...
- 実効 UID ではなく 実 UID で確認する

TOCTOU 競合 (2026 年の更新)

`access()` と `open()` の間に、別のプロセスがファイルを `/etc/passwd` へのリンクにすり替えることができます。安全なのは開いてからエラーを処理する (`EACCES`) か、`RESOLVE_NO_SYMLINKS` 付きの `openat2()` を使うことです。TD8 (`agentd`) はこの理由で `realpath` と `Landlock` を使います。

名前の作成、リンク、削除

C

```
int mknod(const char *path, mode_t mode, dev_t dev);
int mkfifo(const char *path, mode_t mode); /* S_IFIFO */
int link(const char *old, const char *new);
int symlink(const char *target, const char *path);
int unlink(const char *path); /* EISDIR */
int rmdir(const char *path); /* must be emp
```

- `mknod` はどんな種類でも作れます：種類と権限を OR で組み合わせます
(`S_IFREG|S_IRUSR|S_IXUSR` = `S_IFREG|0500`)。現在はほぼデバイス専用です。
ファイルには `open(O_CREAT)`、パイプには `mkfifo`。
- 要求した権限はプロセスの `umask`（通常は `022` か `002`）を通ります。
- `unlink` が消すのはエントリで、ファイルではありません。

開く・作る：open

C

```
int open(const char *path, int flags, ... /* mode_t mode */);  
int openat(int dirfd, const char *path, int flags, ...);
```

- アクセス：O_RDONLY、O_WRONLY、O_RDWR
- 作成：O_CREAT、O_EXCL（存在すれば失敗）、O_TRUNC
- 状態：O_APPEND（各 write を末尾に原子的に）
- 空いている最小のディスクリプタ（ ≥ 0 ）か -1 を返す

現在はさらに

- O_CLOEXEC：exec でディスクリプタを引き継がない（glibc は常に使う。strace で確認）
- O_NOFOLLOW、O_DIRECTORY、O_TMPFILE
- glibc は open を openat(AT_FDCWD, ...) で実装し、creat() は open(..., O_WRONLY|O_CREAT|O_TRUNC) と同じ

close : ファイルはいつ消える？

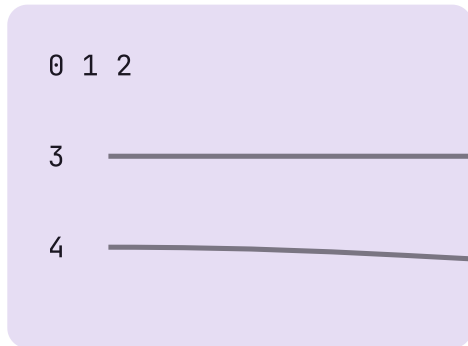
nlink = 0 かつどのディスクリプタも開いていないとき。それまでは、削除されても生きている：

実際の出力

```
$ exec 3< notas.txt; rm notas.txt copia.txt; readlink /proc/$$/fd/3; cat <&3; ls -l atajo; cat atajo
~/fso/tds/tests/tmp/td2/notas.txt (deleted)
hola inodo
lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
cat: atajo: No such file or directory
[exit status 1]
```

カーネル内のファイル管理：3つの表

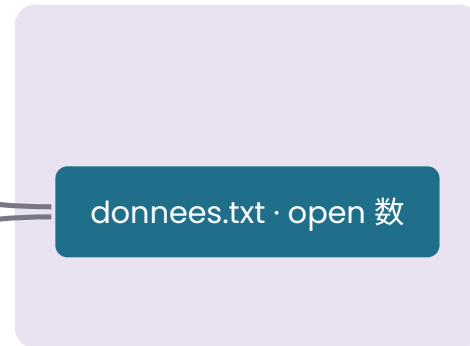
ディスクリプタ（プロセスごと）



オープンファイル（システム）



メモリ上の inode



オフセットは真ん中の表にあります：だから誰が各エントリを共有するかが重要なのです。

エイリアス：2回の `open`、`dup`、`fork`

```
fd = open("donnees.txt", O_RDONLY);
read(fd, buf, 4); write(1, buf, 4);
pid = fork();
/* child */ read(fd, buf, 4); write(1, buf, 4);
/* parent */ wait(NULL); n = read(fd, buf, 4);
```

`read(3)` はエントリ A を使う："efgh" を読み、オフセットは 8 になる

標準出力：
"abcdefgh"

ディスクリプタ（プロセスごと）

親

0	→ 端末
1	→ 端末
2	→ 端末
3	
4	

子

0	→ 端末
1	→ 端末
2	→ 端末
3	
4	

オープンファイル（システム）

エントリ A

O_RDONLY
offset = 8

refs = 2

メモリ上の inode（システム）

donnees.txt

1 個のエントリが開いている

a b c d e f g h

基本の関数、`read` と `write` : 復習

- inode : `stat`、`chmod`、`chown`、`link`、`unlink` ...
- ディスクリプタ : `open`、`read`、`write`、`lseek`、`close`、`fcntl` ...
- `fopen` / `fread` :
ライブラリでありシステムコールではない
(`fileno()`)

`read` は少ないバイト数、終端で 0、-1/ `EINTR` を返すことがあり、`write` は部分的なことがある。スライド 51 の例にはバグがあった：A2.4。

ディレクトリの走査： `opendir`、`readdir`、`closedir`

C

```
DIR *opendir(const char *name);
struct dirent *readdir(DIR *dir); /* NULL at the end */
int closedir(DIR *dir);

struct dirent {
    ino_t      d_ino;      /* inode number */
    unsigned char d_type;  /* DT_REG, DT_DIR, DT_LNK... */
    char       d_name[256];
    /* d_off, d_reclen: internal */
};
```

- `readdir` はエントリをファイルシステム内部の順序（ext4 ではハッシュ索引の順）で返し、アルファベット順ではない
- `.` と `..` も含む
- **更新**：元資料には「最初の 4 つのフィールドは使わない」とありました。現在 `d_ino` と `d_type` は役に立ちます（`stat` を 1 回省ける）が、`d_type` が `DT_UNKNOWN` のこともあり、そのときは `lstat` が必要です。

ディレクトリの走査： `myls -lR`

DIR *d = opendir(dir); の後：

C tds/td2/myls.c

```
struct dirent *e;
while ((e = readdir(d)) != NULL) {
    if (n == cap) {
        cap *= 2;
        names = realloc(names, cap * sizeof *names);
        exit_if(names == NULL, "realloc");
    }
    names[n] = strdup(e->d_name);
    exit_if(names[n] == NULL, "strdup");
    n++;
}
exit_if(closedir(d) == -1, "closedir");
```

実際の出力

```
$ ./myls -lR demo
demo:
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
.
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
..
-rw-rw-r--   1 student student     3 2026-09-30 13:12
a.txt
lrwxrwxrwx   1 student student     5 2026-09-30 13:12
link -> a.txt
prw-rw-r--   1 student student     0 2026-09-30 13:12
pipe
drwxrwxr-x   2 student student   4096 2026-09-30 13:12
sub

demo/sub:
drwxrwxr-x   2 student student   4096 2026-09-30 13:12
.
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
..
```

特別なファイル：チャットの FIFO

サーバー

read() = 0 → 終了

クライアント1

閉じた

クライアント2

閉じた

- ・サーバーが open() から戻る
- ・C1 が FIFO を書き込み用に開く
- ・C2 が FIFO を書き込み用に開く
- ・C1 が閉じる
- ・C2 が閉じる
- ・read() = 0 : 最後の書き手が閉じた → ファイル終端

chat.fifo・カーネルのバッファ : 16 ページ × 4 KiB = 64 KiB



↓ read(fd, buf, 4096) → サーバーの出力

hola a todos
buenas
¿alguien en el lab?
aquí el cliente 2
chao
me voy

read() = 0 → EOF

元資料が語りきれなかったこと

ジャーナリング

ext4 と XFS は
まずジャーナルに
書きます：停電の後でも
ディスク全体を検査する
必要がありません。

コピーオンライト

Btrfs と ZFS は上
書きしません：安価な
スナップショットと全ブ
ロックのチェックサム。

キャッシュと `fsync`

`write` は
ページキャッシュで
終わります。ディスクに
届いたことを保証するのは
`fsync` だけです。

「すべてはファイル」： `/proc`、`/sys`、`/dev`、FUSE は同じ `open` / `read` / `write` の
インタフェースを使います。

解答付き TD2 + AI チャレンジ

- 端末、ディレクトリ、ファイル、パイプで `filetype`
- FIFO チャットを端末 2 つ、次に 3 つで
- `alias_open` と `alias_dup`、`myls -lR`

Bash

```
make -C tds && tds/tests/run.sh td2
```



Universidad
Indoamérica

まとめ

3つの問い

1. ファイルの名前はどこにあり、サイズはどこにある？
2. なぜ `dup` では「abcdefgh」、2 回の `open` では「abcdabcd」と表示される？
3. 削除したファイルのブロックをカーネルはいつ解放する？